République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de Technologie

Compte rendu du TP n° 1

Système de location de vélos et de trottinettes

Matière : **Processus unifié et approches agiles**

Filière : **RT4**

Réalisé par :

Mohamed Gharbi
Mohamed Louay Bourkhis
Soufien Hmaidi
Adem Yengui

RT4/2

Résumé

Nous avons choisi un système de location de vélos et de trottinettes à six cas d'utilisation (UC). Le plus risqué, « Réserver un véhicule », doit empêcher deux clients de réserver le même véhicule sur des créneaux qui se chevauchent. Le rapport suit l'ordre de l'énoncé. Il commence par le cadrage (vision, priorisation, plan des itérations). Il décrit ensuite l'Élaboration, qui se termine sur une première route exécutable et le tag `v0.1-elaboration`, puis la Construction de cet UC : diagrammes UML, code en couches, 44 tests.

Table des matières

1 Choix du système **2**

2 Vision et acteurs (question 1) **2**
2.1 Vision en cinq lignes 2
2.2 Acteurs 2

3 Liste des cas d'utilisation (question 2) **3**

4 Priorisation des UC (question 3) **3**

5 Planification : itérations, phases et jalons (question 4) **4**

6 Partie 1 : Élaboration **4**
6.1 Dépôt, tableau de bord, tickets, milestones, branches et tags 4
6.2 Diagramme de cas d'utilisation (PlantUML) 5
6.3 Architecture de base exécutable 5
6.4 Tag `v0.1-elaboration` et fermeture du milestone 7

7 Partie 2 : Construction et Transition **7**
7.1 Fiche UC2, diagramme de classes et diagramme de séquence 7
7.2 Implémentation en couches (métier et accès aux données) 8
7.3 Tests unitaires et test d'intégration dérivés du scénario nominal 11
7.4 Revue croisée 12

8 Traçabilité UC → ticket → branche → classes → tests **13**

9 Bilan, limites et suite **13**

A Commandes utiles **13**

1 Choix du système

Le système gère la location de vélos et de trottinettes en stations fixes. Il compte six cas d'utilisation. L'UC à risque technique est UC2, Réserver un véhicule : deux demandes peuvent viser le même véhicule sur des créneaux qui se chevauchent, parfois au même instant.

Choix techniques.

— Langage et framework : Python 3.13, FastAPI 0.142.2.

— Accès aux données : SQLAlchemy 2.1.3 sur SQLite.

— Tests : pytest 9.1.1 (client de test HTTP via `httpx2`).

— Modélisation : PlantUML ; intégration continue : GitHub Actions.

— Gestion de version : Git (une branche par UC, jalons de phase sous forme de tags).

2 Vision et acteurs (question 1)

2.1 Vision en cinq lignes

1. **Problème** : louer un vélo ou une trottinette en station se fait sans garantie de disponibilité ; doubles réservations et manque de visibilité frustrent les usagers.
2. **Utilisateurs** : clients (étudiants, salariés, touristes) et opérateurs de flotte.
3. **Valeur client** : réserver en ligne un véhicule sur un créneau garanti sans conflit, payer et restituer simplement.
4. **Valeur opérateur** : visibilité sur la flotte, moins de litiges.
5. **Périmètre** : API web, stations fixes, paiement via une passerelle externe simulée.

2.2 Acteurs

Acteur	Type	Rôle
Client	principal	consulte, réserve, paie, annule, restitue
Opérateur de flotte	principal	gère les véhicules (ajout, retrait, maintenance)
Passerelle de paiement	secondaire (système externe)	autorise et capture les paiements

Table 1 – Acteurs du système

3 Liste des cas d'utilisation (question 2)

N°	**Cas d'utilisation**	**Acteur(s)**	**Description**
UC1	Consulter les véhicules disponibles	Client	lister les véhicules libres d'une station sur un créneau
UC2	Réserver un véhicule	Client	bloquer un véhicule sur un créneau (UC à risque : conflits de réservation)
UC3	Payer la location	Client, Passerelle	régler le montant via la passerelle externe
UC4	Annuler une réservation	Client	libérer un créneau réservé
UC5	Restituer le véhicule	Client	clore la location, calculer le prix final (inclut UC3)
UC6	Gérer la flotte	Opérateur	ajouter, retirer, mettre en maintenance un véhicule

Table 2 – Cas d'utilisation

4 Priorisation des UC (question 3)

Chaque UC reçoit une note de 1 (faible) à 3 (fort) pour la valeur métier (V), le risque (R) et l'impact sur l'architecture (A).

UC	**V**	**R**	**A**	**Total**	**Itération**
UC2 Réserver un véhicule	3	3	3	**9**	It1 (Élaboration)
UC3 Payer la location	3	3	2	8	It2
UC1 Consulter les disponibilités	3	1	2	6	It1
UC5 Restituer le véhicule	2	2	2	6	It2
UC6 Gérer la flotte	2	1	2	5	It3
UC4 Annuler une réservation	2	1	1	4	It3

Table 3 – Priorisation selon valeur métier, risque et impact architectural

UC à traiter en premier. UC2 a la note maximale sur les trois critères. C'est lui qui fixe le modèle de données, la gestion des transactions et la parade aux conflits, et les autres UC s'appuient dessus. L'UP demande de traiter d'abord ce qui est risqué et structurant pour l'architecture : UC2 passe donc en Élaboration. UC3 suit, parce qu'il dépend d'un système externe, la passerelle de paiement.

5 Planification : itérations, phases et jalons (question 4)

Phase	Itér.	Contenu	Jalon	Tag
Lancement	cadrage	vision, acteurs, UC, priorisation, plan	LCO	`v0.0-lancement`
Élaboration	It1	architecture de base exécutable : `POST /reservations` de bout en bout (UC2), CI verte	LCA	`v0.1-elaboration`
Construction	It2	UC2 complet (règles, alternatives, concurrence), UC3, UC5	aucun	aucun
Construction	It3	UC4, UC6	IOC	`v0.2-construction`
Transition	fin It3	revue croisée, recette, documentation de déploiement	PR	`v1.0-transition`

Table 4 – Phases UP, itérations et jalons

Signification des jalons :

- **LCO** (*Lifecycle Objectives*) : objectifs du cycle de vie validés ;
- **LCA** (*Lifecycle Architecture*) : architecture de base stabilisée et *exécutable* ;
- **IOC** (*Initial Operational Capability*) : capacité opérationnelle initiale ;
- **PR** (*Product Release*) : livraison du produit.

Les trois itérations :

- **It1 (Élaboration)** : lever le risque technique ; livrer une route de bout en bout contrôleur → service → accès aux données pour UC2, avec un premier test automatisé en CI.
- **It2 (Construction)** : UC2 complet (fiche, classes, séquence, tests), puis UC3 et UC5.
- **It3 (Construction → Transition)** : UC4, UC6, revue croisée, stabilisation, livraison.

6 Partie 1 : Élaboration

6.1 Dépôt, tableau de bord, tickets, milestones, branches et tags

Le dépôt Git existe en local. Le script `github/setup-github.sh` le publie sur GitHub une fois le dépôt distant créé : il crée les milestones, les tickets et le tableau de bord, puis pousse les branches et les tags.

Tickets. Chaque UC a son ticket (fichiers `github/issues/*.md`). Trois tickets de plus suivent les livrables de phase :

Ticket	Titre	Milestone
#1 à #6	UC1 à UC6	Construction
#7	Lancement : vision, UC, priorisation, plan	Lancement
#8	Élaboration : architecture de base exécutable (UC2)	Élaboration
#9	Transition : revue croisée et livraison	Transition

Table 5 – Tickets et milestones

Milestones. Un milestone par phase UP : Lancement, Élaboration, Construction, Transition. Les tickets portent les labels `cas-utilisation`, `risque-technique` ou `phase`.

Tableau de bord. Le script crée un *GitHub Project* et y range tous les tickets.

Branches. Chaque UC possède sa branche :

- `uc1-consulter-disponibilites`
- `uc2-reserver-vehicule`
- `uc3-payer-location`
- `uc4-annuler-reservation`
- `uc5-restituer-vehicule`
- `uc6-gerer-flotte`

Tags. Chaque jalon de phase est un tag Git annoté. Nous avons posé `v0.0-lancement` (LCO) et `v0.1-elaboration` (LCA) ; `v0.2-construction` et `v1.0-transition` viendront aux jalons suivants.

6.2 Diagramme de cas d'utilisation (PlantUML)

La figure 1 montre les trois acteurs et les six UC. UC5 inclut UC3 : rendre le véhicule déclenche le paiement. Source : `docs/uml/cas-utilisation.puml`.

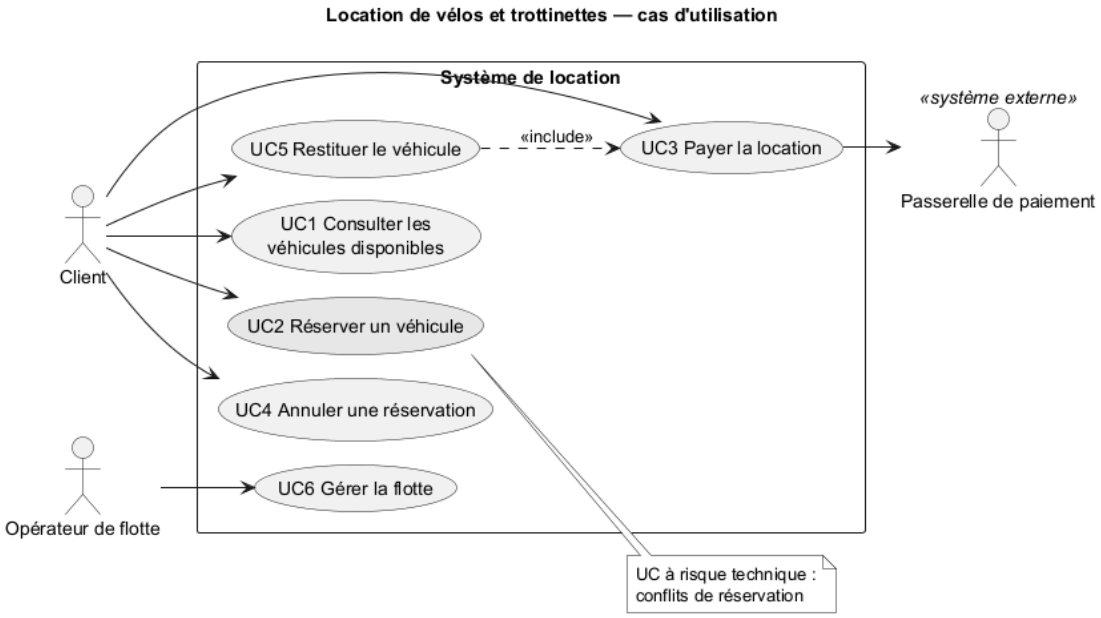


FIGURE 1 – Diagramme de cas d'utilisation

6.3 Architecture de base exécutable

Le code est découpé en quatre couches :

Couche	Emplacement	Rôle
Contrôleur	`app/api/reservations.py`	routes FastAPI, schémas Pydantic, HTTP ↔ service
Service	`app/services/`	orchestration de l'UC, règles, transaction ; dépend de ports (`Protocol`)
Métier	`app/domain/`	objets purs : `Creneau`, tarif, énumérations, erreurs
Accès aux données	`app/repositories/`, `app/db.py`, `app/models.py`	dépôts et entités SQLAlchemy sur SQLite

Table 6 – Architecture en couches

Route de l'UC à risque. `POST /reservations` passe par toutes les couches. Le contrôleur reçoit la demande. `ReservationService` vérifie que le client et le véhicule existent, puis que le créneau est libre. Le dépôt SQL enregistre la réservation.

Traitement du risque technique. La recherche de chevauchement et l'insertion se font dans la même transaction. Celle-ci commence par `BEGIN IMMEDIATE`, qui prend tout de suite le verrou d'écriture de SQLite. Si deux demandes arrivent en même temps, la seconde attend la fin de la première, puis voit sa réservation et la refuse.

Listing 1 – `app/db.py` : transaction `BEGIN IMMEDIATE`

```python
def creer_engine(url: str) -> Engine:
    engine = create_engine(url, connect_args={"check_same_thread": False, "timeout": 30})

    @event.listens_for(engine, "connect")
    def _desactiver_begin_implicite(connexion_dbapi, _enregistrement):
        connexion_dbapi.isolation_level = None

    @event.listens_for(engine, "begin")
    def _begin_immediate(connexion):
        connexion.exec_driver_sql("BEGIN IMMEDIATE")

    return engine
```

Premier test automatisé. Le premier test d'intégration,

`test_reserver_creneau_libre_puis_conflit`,

vérifie qu'un créneau libre donne `201`, puis que le même créneau demandé par un autre client donne `409 CONFLIT_RESERVATION`.

Listing 2 – Premier test d'intégration

```python
def test_reserver_creneau_libre_puis_conflit(client_http):
    premiere = client_http.post(
        "/reservations", json={"client_id": 1, "vehicule_id": 1, **CRENEAU_DEMAIN}
    )
    assert premiere.status_code == 201
    assert premiere.json()["statut"] == "CONFIRMEE"

    seconde = client_http.post(
```

```
        "/reservations", json={"client_id": 2, "vehicule_id": 1, **CRENEAU_DEMAIN}
    )
    assert seconde.status_code == 409
    assert seconde.json()["erreur"] == "CONFLIT_RESERVATION"
```

Intégration continue. GitHub Actions lance les tests à chaque *push* et à chaque *pull request*.

Listing 3 – `.github/workflows/ci.yml`

```
name: CI

on:
  push:
  pull_request:

jobs:
  tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt
      - run: pytest -v
```

Nous avons aussi lancé l'application avec `uvicorn` et envoyé deux requêtes à la main : la première réservation renvoie `201`, la seconde, sur le même créneau, renvoie `409`.

6.4 Tag `v0.1-elaboration` et fermeture du milestone

Nous avons fusionné `uc2-reserver-vehicule` dans `main` (*merge* `-no-ff`, message « Closes #8, refs #2 »). Les 13 tests passaient sur `main`, et nous avons posé le tag annoté `v0.1-elaboration`. Le fichier `github/milestones.md` marque les milestones Lancement et Élaboration comme fermés ; le script les fermera sur GitHub. Le jalon LCA correspond donc à du code qui s'exécute et dont les tests passent.

7 Partie 2 : Construction et Transition

7.1 Fiche UC2, diagramme de classes et diagramme de séquence

Fiche UC2 : Réserver un véhicule

Acteur principal	Client
Parties prenantes	Client (créneau garanti) ; Opérateur (pas de double réservation)
Préconditions	le client est identifié ; le véhicule existe
Garanties en cas de succès	une réservation `CONFIRMEE` est enregistrée avec son montant estimé ; aucune autre réservation confirmée ne chevauche ce créneau pour ce véhicule ni pour ce client
Garanties minimales	en cas d'échec, rien n'est enregistré
Déclencheur	le client choisit un véhicule (issu de UC1) et un créneau

Table 7 – En-tête de la fiche UC2

Scénario nominal.

1. Le client envoie sa demande : identifiant client, identifiant véhicule, début, fin.

2. Le système vérifie le créneau (début avant la fin, durée $\leq$ 24 h, début dans le futur).
3. Le système vérifie que le client et le véhicule existent.
4. Le système vérifie que le véhicule n'est pas en maintenance.
5. Le système vérifie qu'aucune réservation confirmée ne chevauche le créneau pour ce véhicule.
6. Le système vérifie que le client n'a pas déjà une réservation confirmée sur ce créneau.
7. Le système calcule le montant estimé (vélo 2 euros/h, trottinette 3 euros/h, toute heure entamée est due).
8. Le système enregistre la réservation `CONFIRMEE` et la renvoie (HTTP 201).

Scénarios alternatifs.

— **2a** Début $\geq$ fin ou durée $>$ 24 h $\rightarrow$ `CRENEAU_INVALIDE` (422).
— **2b** Début $\leq$ maintenant $\rightarrow$ `CRENEAU_DANS_LE_PASSE` (422).
— **3a** Client ou véhicule inconnu $\rightarrow$ `CLIENT_INTROUVABLE` / `VEHICULE_INTROUVABLE` (404).
— **4a** Véhicule en maintenance $\rightarrow$ `VEHICULE_INDISPONIBLE` (409).
— **5a** Créneau déjà réservé pour ce véhicule $\rightarrow$ `CONFLIT_RESERVATION` (409).
— **6a** Le client a déjà une réservation qui chevauche $\rightarrow$ `CONFLIT_CLIENT` (409).

Règles métier. Les étapes 2 à 6 du scénario appliquent les règles suivantes :

Règle	Énoncé
R1	`debut < fin` et durée $\leq$ 24 h
R2	`debut` strictement après l'instant courant
R3	le client et le véhicule existent
R4	le véhicule n'est pas `EN_MAINTENANCE`
R5	pas de chevauchement avec une réservation `CONFIRMEE` du véhicule
R6	pas de chevauchement avec une réservation `CONFIRMEE` du client

Table 8 – Règles métier de UC2

Deux créneaux a et b se chevauchent si et seulement si $a.debut < b.fin$ et $b.debut < a.fin$. Un créneau de 10 h à 11 h et un créneau de 11 h à 12 h ne se chevauchent donc pas. Une date reçue sans fuseau horaire est lue en UTC, et l'API répond en UTC (suffixe `Z`).

Diagramme de classes

La figure 2 range les classes par paquetage : contrôleur, services, métier, accès aux données et entités persistantes. Le service ne voit les dépôts qu'à travers des interfaces.

Diagramme de séquence (scénario nominal et alternative 5a)

La figure 3 suit une réservation du `POST` jusqu'au `COMMIT`, puis montre l'alternative 5a : créneau déjà pris, annulation de la transaction et réponse 409.

7.2 Implémentation en couches (métier et accès aux données)

Couche métier. L'objet valeur `Creneau` convertit les dates en UTC, applique R1 et sait dire si deux créneaux se chevauchent. Le module `tarif` calcule le montant en centimes entiers, pour éviter les erreurs d'arrondi des nombres flottants.

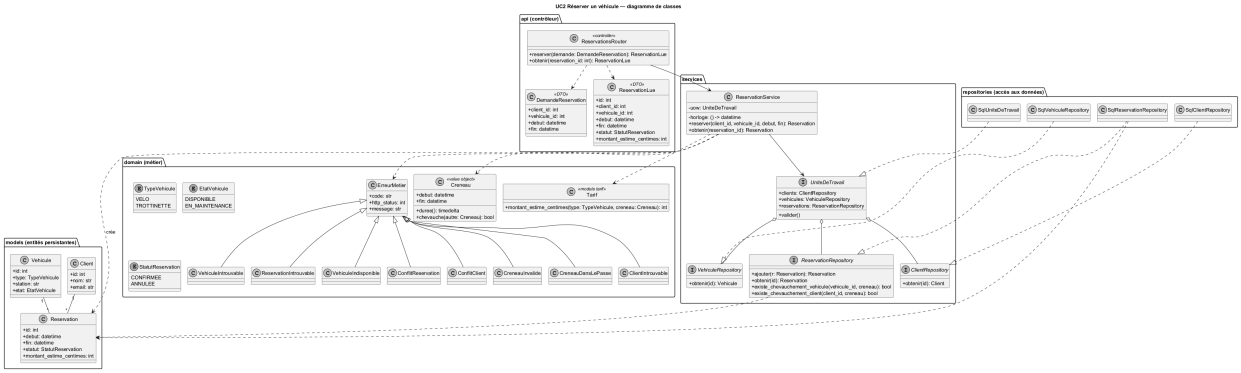


FIGURE 2 – Diagramme de classes de UC2 (`docs/uml/classes-uc2.puml`)

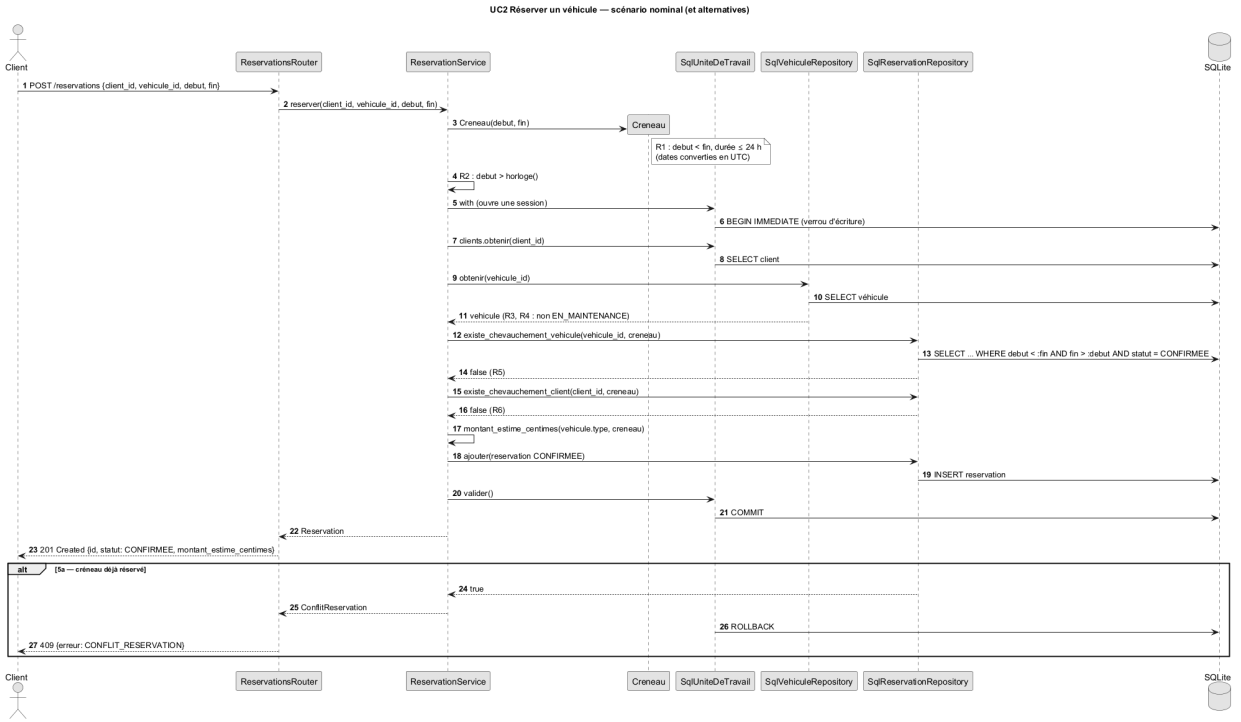


FIGURE 3 – Diagramme de séquence de UC2 (`docs/uml/sequence-uc2.puml`)

Listing 4 – `app/domain/creneau.py` (extrait)

```python
@dataclass(frozen=True)
class Creneau:
    debut: datetime
    fin: datetime

    def __post_init__(self) -> None:
        object.__setattr__(self, "debut", en_utc_naif(self.debut))
        object.__setattr__(self, "fin", en_utc_naif(self.fin))
        if self.debut >= self.fin:
            raise CreneauInvalide("Le début du créneau doit précéder sa fin.")
        if self.duree > DUREE_MAX:
            raise CreneauInvalide("Un créneau ne peut pas dépasser 24 heures.")

    def chevauche(self, autre: "Creneau") -> bool:
        return self.debut < autre.fin and autre.debut < self.fin
```

Couche service. `ReservationService` applique les règles R1 à R6 dans l'ordre de la fiche UC. Il reçoit une *unité de travail* abstraite et une horloge : les tests unitaires lui passent des faux dépôts en mémoire et une heure fixe, sans base de données.

Listing 5 – `app/services/reservation_service.py` (extrait)

```python
def reserver(self, client_id, vehicule_id, debut, fin) -> Reservation:
    creneau = Creneau(debut, fin)  # R1
    if creneau.debut <= self._horloge():  # R2
        raise CreneauDansLePasse("Le créneau doit commencer dans le futur.")
    with self._uow as uow:
        if uow.clients.obtenir(client_id) is None:  # R3
            raise ClientIntrouvable(f"Client {client_id} introuvable.")
        vehicule = uow.vehicules.obtenir(vehicule_id)
        if vehicule is None:  # R3
            raise VehiculeIntrouvable(f"Véhicule {vehicule_id} introuvable.")
        if vehicule.etat == EtatVehicule.EN_MAINTENANCE:  # R4
            raise VehiculeIndisponible("Le véhicule est en maintenance.")
        if uow.reservations.existe_chevauchement_vehicule(vehicule_id, creneau):  # R5
            raise ConflitReservation("Le véhicule est déjà réservé sur ce créneau.")
        if uow.reservations.existe_chevauchement_client(client_id, creneau):  # R6
            raise ConflitClient("Vous avez déjà une réservation sur ce créneau.")
        reservation = Reservation(
            client_id=client_id, vehicule_id=vehicule_id,
            debut=creneau.debut, fin=creneau.fin,
            statut=StatutReservation.CONFIRMEE,
            montant_estime_centimes=montant_estime_centimes(vehicule.type, creneau),
        )
        uow.reservations.ajouter(reservation)
        uow.valider()
        return reservation
```

Couche d'accès aux données. Le dépôt SQL écrit la règle de chevauchement sous forme de requête. Les réservations annulées n'y comptent pas.

Listing 6 – `app/repositories/depots.py` (extrait)

```python
def _existe_chevauchement(self, critere, creneau: Creneau) -> bool:
    requete = (
        select(Reservation.id)
        .where(
            critere,
            Reservation.statut == StatutReservation.CONFIRMEE,
            Reservation.debut < creneau.fin,
            Reservation.fin > creneau.debut,
```

```
        )
        .limit(1)
    )
    return self._session.scalar(requete) is not None
```

API. L'API a deux routes. `POST /reservations` répond 201, 404, 409 ou 422 ; `GET /reservations/{id}` répond 200, 404 ou 422. Une erreur métier renvoie :

```
{"erreur": "<CODE>", "message": "..."}
```

7.3 Tests unitaires et test d'intégration dérivés du scénario nominal

Nous avons travaillé en TDD : chaque test est écrit avant le code qu'il couvre. On le lance une première fois pour le voir échouer, puis on écrit juste assez de code pour qu'il passe.

Fichier	Tests	Objet
`tests/unit/test_creneau.py`	11	validation, chevauchement, créneaux contigus, durée max, conversion UTC
`tests/unit/test_tarif.py`	6	heure entamée due, bornes (1 h + 1 µs facturée 2 h)
`tests/unit/test_reservation_service.py`	13	scénario nominal et règles R1 à R6 avec faux dépôts en mémoire
`tests/integration/test_depots.py`	5	requêtes SQL de chevauchement, réservations annulées ignorées
`tests/integration/test_api_reservations.py`	8	scénario nominal HTTP, erreurs, fuseaux horaires, entrées hors limites
`tests/integration/test_concurrence.py`	1	deux réservations simultanées : un seul succès
Total	**44**	tous passent

TABLE 9 – Répartition des tests

Test unitaire du scénario nominal. Le test appelle le service avec les faux dépôts et une heure fixe. Il vérifie la réservation renvoyée, son montant et l'appel à `valider()`.

Listing 7 – Test unitaire du scénario nominal

```
def test_scenario_nominal_enregistre_une_reservation_confirmee(service, uow):
    reservation = service.reserver(ALICE, TROTTINETTE, h(10), h(11, 30))

    assert reservation.id == 1
    assert reservation.statut == StatutReservation.CONFIRMEE
    assert reservation.montant_estime_centimes == 600
    assert uow.reservations.elements == [reservation]
    assert uow.valide
```

Test d'intégration du scénario nominal. Il passe par toutes les couches jusqu'à une base SQLite sur disque. Le client réserve (réponse 201 avec le montant), puis relit la même réservation par `GET`.

Listing 8 – Test d'intégration du scénario nominal

```
def test_scenario_nominal_reserver_puis_consulter(client_http):
```

```
    reponse = client_http.post(
        "/reservations",
        json={"client_id": 1, "vehicule_id": 2,
              "debut": "2026-10-08T10:00:00", "fin": "2026-10-08T11:30:00"},
    )
    assert reponse.status_code == 201
    reservation = reponse.json()
    assert reservation == {
        "id": 1, "client_id": 1, "vehicule_id": 2,
        "debut": "2026-10-08T10:00:00Z", "fin": "2026-10-08T11:30:00Z",
        "statut": "CONFIRMEE", "montant_estime_centimes": 600,
    }

    consultee = client_http.get(f"/reservations/{reservation['id']}")
    assert consultee.status_code == 200
    assert consultee.json() == reservation
```

Test de concurrence (risque technique). Deux fils d'exécution réservent le même créneau au même moment, dix fois de suite. À chaque essai, l'un doit réussir et l'autre recevoir un conflit. Pour vérifier que ce test détecte bien le problème, nous avons remplacé `BEGIN IMMEDIATE` par `BEGIN` : le test échoue (`database is locked`). Il repasse dès que le verrou est remis. Pendant la relecture, un essai sans verrou a donné une double réservation dans 27 cas sur 28.

7.4 Revue croisée

Support fourni à l'équipe relectrice.

- `docs/revue/checklist-revue-croisee.md` : grille couvrant la traçabilité, le respect des couches, le traitement du risque de conflit et la qualité, avec un tableau de remarques à compléter ;
- `docs/revue/pr-uc2.md` : description de la *pull request* `uc2-reserver-vehicule` → `main`.

Première relecture et corrections. Avant la revue croisée, toute la branche a été relue. La relecture n'a trouvé aucun défaut critique et trois défauts importants. Nous avons corrigé chacun en écrivant d'abord un test qui échouait :

Constat	Correction
Identifiants $\geq 2^{63}$ et dates extrêmes avec fuseau provoquaient une erreur 500	identifiants bornés (1 à $2^{63}-1$) et date hors limites → réponse 422
Dates des réponses en UTC sans indication de fuseau	sérialisation avec le suffixe `Z`
Checklist et diagramme de classes masquaient la dépendance du service vers les entités	documentation et diagramme alignés sur le code

TABLE 10 – Corrections issues de la première relecture

L'autre équipe relit la *pull request* avec la grille et note ses remarques dans le tableau de la checklist.

8 Traçabilité UC → ticket → branche → classes → tests

Maillon	Artefact pour UC2
UC	`docs/fiches/UC2-reserver-vehicule.md`, `docs/uml/cas-utilisation.puml`
Ticket	#2 pour UC2 (milestone Construction) ; #8 pour l'Élaboration
Branche	`uc2-reserver-vehicule`
Classes	`docs/uml/classes-uc2.puml`, `docs/uml/sequence-uc2.puml` ; code dans `app/`
Tests	`tests/unit/test_reservation_service.py`, `tests/integration/test_api_reservations.py`, `tests/integration/test_concurrence.py`

Table 11 – Chaîne de traçabilité de UC2

Historique Git. Le graphe des commits montre deux fusions de la branche de UC2 dans `main` : la première à la fin de l'Élaboration (« Closes #8, refs #2 », suivie du tag `v0.1-elaboration`), la seconde à la fin de la Construction de UC2 (« Closes #2 »). Tous les commits de code de la branche citent le ticket (« Refs #2 »).

9 Bilan, limites et suite

Bilan. Le tag `v0.1-elaboration` marque une route de réservation qui fonctionne et dont le test passe en CI. UC2 est maintenant complet : 44 tests passent. Le test de concurrence échoue dès qu'on retire le verrou, ce qui montre qu'il couvre bien le risque de double réservation.

Limites connues.

- L'authentification est hors périmètre : l'identifiant client est fourni dans la requête et la consultation n'est pas restreinte au propriétaire.
- Les lectures prennent aussi le verrou d'écriture SQLite (acceptable à l'échelle du TP).
- Le script GitHub suppose un dépôt distant neuf (numérotation des tickets de #1 à #9).

Suite. Publication sur GitHub (dépôt, tickets, milestones, tableau de bord, branches et tags), exécution de la CI, revue croisée par une autre équipe, puis UC3 et UC5 (It2), UC4 et UC6 (It3), et pose des tags `v0.2-construction` et `v1.0-transition`.

A Commandes utiles

Listing 9 – Installation, lancement et tests

```
python -m venv .venv
.venv/Scripts/python -m pip install -r requirements.txt
.venv/Scripts/python -m app.seed
.venv/Scripts/python -m uvicorn app.main:creer_app --factory --reload
.venv/Scripts/python -m pytest -v
```

Listing 10 – Publication sur GitHub

```
git remote add origin <url>
bash github/setup-github.sh <owner>/<repo>
```